

Лабораторная работа № 14

Архитектура компьютера, раздел Операционные системы

Бондарь Т. В.

Российский университет дружбы народов, Москва, Россия

Информация

- Бондарь Татьяна Владимировна
- НКАбд-01-24, студ. билет №1132246711
- Российский университет дружбы народов
- https://github.com/tvbondar/study_2024-2025_os-intro

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Задание

1. Написать командный файл, реализующий упрощённый механизм семафоров. Командный файл должен в течение некоторого времени t_1 дожидаться освобождения ресурса, выдавая об этом сообщение, а дождавшись его освобождения, использовать его в течение некоторого времени $t_2 < t_1$, также выдавая информацию о том, что ресурс используется соответствующим командным файлом (процессом). Запустить командный файл в одном виртуальном терминале в фоновом режиме, перенаправив его вывод в другой (`> /dev/tty#`, где `#` — номер терминала куда перенаправляется вывод), в котором также запущен этот файл, но не фоновом, а в привилегированном режиме. Доработать программу так, чтобы имелась возможность взаимодействия трёх и более процессов.

2. Реализовать команду `man` с помощью командного файла. Изучите содержимое каталога `/usr/share/man/man1`. В нем находятся архивы текстовых файлов, содержащих справку по большинству установленных в системе программ и команд. Каждый архив можно открыть командой `less` сразу же просмотрев содержимое справки. Командный файл должен получать в виде аргумента командной строки название команды и в виде результата выдавать справку об этой команде или сообщение об отсутствии справки, если соответствующего файла нет в каталоге `man1`.

3. Используя встроенную переменную \$RANDOM, напишите командный файл, генерирующий случайную последовательность букв латинского алфавита. Учтите, что \$RANDOM выдаёт псевдослучайные числа в диапазоне от 0 до 32767.

Теоретическое введение

Командный процессор (командная оболочка, интерпретатор команд shell) — это программа, позволяющая пользователю взаимодействовать с операционной системой компьютера. В операционных системах типа UNIX/Linux наиболее часто используются следующие реализации командных оболочек: – оболочка Борна (Bourne shell или sh) — стандартная командная оболочка UNIX/Linux, содержащая базовый, но при этом полный набор функций; – C-оболочка (или csh) — надстройка на оболочкой Борна, использующая C-подобный синтаксис команд с возможностью сохранения истории выполнения команд; – оболочка Корна (или ksh) — напоминает оболочку C, но операторы управления программой совместимы с операторами оболочки Борна;

– BASH — сокращение от Bourne Again Shell (опять оболочка Борна), в основе своей совмещает свойства оболочек C и Корна (разработка компании Free Software Foundation). POSIX (Portable Operating System Interface for Computer Environments) — набор стандартов описания интерфейсов взаимодействия операционной системы и прикладных программ. Стандарты POSIX разработаны комитетом IEEE (Institute of Electrical and Electronics Engineers) для обеспечения совместимости различных UNIX/Linux-подобных операционных систем и переносимости прикладных программ на уровне исходного кода. POSIX-совместимые оболочки разработаны на базе оболочки Корна.

Выполнение лабораторной работы

1. Написать командный файл, реализующий упрощённый механизм семафоров. Командный файл должен в течение некоторого времени t_1 дожидаться освобождения ресурса, выдавая об этом сообщение, а дождавшись его освобождения, использовать его в течение некоторого времени $t_2 < t_1$, также выдавая информацию о том, что ресурс используется соответствующим командным файлом (процессом). Запустить командный файл в одном виртуальном терминале в фоновом режиме, перенаправив его вывод в другой (`> /dev/tty#`, где `#` — номер терминала куда перенаправляется вывод), в котором также запущен этот файл, но не фоновом, а в привилегированном режиме. Доработать программу так, чтобы имелась возможность взаимодействия трёх и более процессов. (рис. (fig:001?)). (рис. (fig:002?)).

```
#!/bin/bash

lockfile="./lock.file"
exec {fn}>${lockfile}

while test -f "$lockfile"
do
if flock -n ${fn}
then
    echo "File is_blocked"
    sleep 5
    echo "File is unlocked"
    flock -u ${fn}
else
    echo "File is blocked"
    sleep 5
fi
done
```

```
tvbondar@fedora:~$ chmod +x lab14-1.sh
tvbondar@fedora:~$ bash lab14-1.sh
File is blocked
File is unlocked
File is blocked
File is unlocked
File is blocked
```

Рис. 2: 2

2. Реализовать команду `man` с помощью командного файла. Изучите содержимое каталога `/usr/share/man/man1`. В нем находятся архивы текстовых файлов, содержащих справку по большинству установленных в системе программ и команд. Каждый архив можно открыть командой `less` сразу же просмотрев содержимое справки. Командный файл должен получать в виде аргумента командной строки название команды и в виде результата выдавать справку об этой команде или сообщение об отсутствии справки, если соответствующего файла нет в каталоге `man1`. (рис. (fig:003?)). (рис. (fig:004?)). (рис.

```

gitpath-1.gz      kitten-diff-1.gz      nrm-rm-ncu-1.gz      rpath-1.gz        ps-diff-1.gz
getbookids-1.gz  kitten-disable-1.gz   nrm-rm-nvif-1.gz     rx-1.gz           ps-change-log-1.gz
gettext-1.gz     kitten-edit-in-vim-1.gz nrm-rm-register-1.gz  sdr-1.gz          stat-1.gz
ghdoc-1.gz       kitten-q-nm-1.gz      nrm-rm-rename-1.gz   sdrpr-1.gz        stmp-1.gz
ghgroup-1.gz     kitten-q-focus-tab-1.gz nrm-rm-report-1.gz    sh-1.gz           stlff-1.gz
ghpage-1.gz      kitten-q-focus-window-1.gz nrm-rm-1.gz          scalar-1.gz        stlty-1.gz
gh-1.gz          kitten-q-get-calls-1.gz nrm-rm-ncu-1.gz       scanmap-1.gz       tface-1.gz
gh-alias-1.gz    kitten-q-get-text-1.gz nrm-rm-alias-log-1.gz  scmap-1.gz        tprep-1.gz
gh-alias-delete-1.gz kitten-q-get-1.gz      nrm-rm-alias-log-1.gz  scmap-1.gz        tpr-1.gz
gh-alias-import-1.gz kitten-q-help-1.gz     nrm-rm-alias-log-1.gz  scmap-1.gz        tpr-1.gz
gh-alias-list-1.gz kitten-q-hyperlinked-grap-1.gz nrm-rm-alias-log-1.gz  scmap-1.gz        tpr-1.gz
gh-alias-set-1.gz kitten-ls-1.gz        nrm-rm-alias-log-1.gz  scmap-1.gz        tpr-1.gz
gh-alias-1.gz    kitten-q-kitten-1.gz  nrm-rm-alias-log-1.gz  scmap-1.gz        tpr-1.gz
gh-attestation-1.gz kitten-q-ls-1.gz      nrm-rm-alias-log-1.gz  scmap-1.gz        tpr-1.gz
gh-attestation-download-1.gz kitten-q-launch-1.gz  nrm-rm-alias-log-1.gz  scmap-1.gz        tpr-1.gz
gh-attestation-trusted-root-1.gz kitten-q-launch-1.gz  nrm-rm-alias-log-1.gz  scmap-1.gz        tpr-1.gz
gh-attestation-verify-1.gz kitten-q-launch-1.gz  nrm-rm-alias-log-1.gz  scmap-1.gz        tpr-1.gz
gh-auth-1.gz     kitten-q-launch-1.gz  nrm-rm-alias-log-1.gz  scmap-1.gz        tpr-1.gz
gh-auth-login-1.gz kitten-q-launch-1.gz  nrm-rm-alias-log-1.gz  scmap-1.gz        tpr-1.gz
gh-auth-logout-1.gz kitten-q-launch-1.gz  nrm-rm-alias-log-1.gz  scmap-1.gz        tpr-1.gz
gh-auth-refresh-1.gz kitten-q-launch-1.gz  nrm-rm-alias-log-1.gz  scmap-1.gz        tpr-1.gz
gh-auth-set-1.gz kitten-q-launch-1.gz  nrm-rm-alias-log-1.gz  scmap-1.gz        tpr-1.gz
gh-auth-status-1.gz kitten-q-launch-1.gz  nrm-rm-alias-log-1.gz  scmap-1.gz        tpr-1.gz

```

Рис. 3: 3


```
#!/bin/bash

a=$1
if test -f "/usr/share/man/man1/$a.1.gz"
then less /usr/share/man/man1/$a.1.gz
else
echo "There is no such command"
fi
```

Рис. 4: 4

[illegible]

Рис. 5: 5

```
tvbondar@fedora:~$ chmod +x lab14-2.sh  
tvbondar@fedora:~$ bash lab14-2.sh kill  
tvbondar@fedora:~$
```

Рис. 6: 6

3. Используя встроенную переменную \$RANDOM, напишите командный файл, генерирующий случайную последовательность букв латинского алфавита. Учтите, что \$RANDOM выдаёт псевдослучайные числа в диапазоне от 0 до 32767. (рис. (fig:007?)). (рис. (fig:008?)).

```
#! /bin/bash

a=$1

for ((i=0; i<$a; i++))
do
    ((char=$RANDOM%26+1))
    case $char in
        1) echo -n a;; 2) echo -n b;; 3) echo -n c;; 4) echo -n d;; 5) echo -n e;; 6) echo -n f;;
        7) echo -n g;; 8) echo -n h;; 9) echo -n i;; 10) echo -n j;; 11) echo -n k;; 12) echo -n l;;
        13) echo -n m;; 14) echo -n n;; 15) echo -n o;; 16) echo -n p;; 17) echo -n r;; 18) echo -n s;;
        19) echo -n t;; 20) echo -n q;; 21) echo -n u;; 22) echo -n v;;
        23) echo -n w;; 24) echo -n x;; 25) echo -n y;; 26) echo -n z;;
    esac
done
echo
```

Рис. 7: 7

```
tvbondar@fedora:~$ chmod +x lab14-3.sh
tvbondar@fedora:~$ bash lab14-3.sh 14
kyqxjuycethnwjd
tvbondar@fedora:~$
```

Рис. 8: 8

Контрольные вопросы

1. Найдите синтаксическую ошибку в следующей строке: `1 while [$1 != "exit"]` В данной строчке допущены следующие ошибки: не хватает пробелов после первой скобки [и перед второй скобкой] выражение \$1 необходимо взять в "", потому что эта переменная может содержать пробелы Таким образом, правильный вариант должен выглядеть так:
`while [ "$1" != "exit" ]`
2. Как объединить (конкатенация) несколько строк в одну? Чтобы объединить несколько строк в одну, можно воспользоваться несколькими способами: Первый: `VAR1="Hello,"`
`VAR2=" World" VAR3="VAR1VAR2" echo`
`"VAR3" : Hello, World : VAR1 = "Hello, "VAR1+ = "World"echo"VAR1"`
Результат: Hello, World

3. Найдите информацию об утилите seq. Какими иными способами можно реализовать её функционал при программировании на bash? Команда seq в Linux используется для генерации чисел от ПЕРВОГО до ПОСЛЕДНЕГО шага INCREMENT. Параметры: seq LAST: если задан только один аргумент, он создает числа от 1 до LAST с шагом шага, равным 1. Если LAST меньше 1, значение is не выдает. seq FIRST LAST: когда заданы два аргумента, он генерирует числа от FIRST до LAST с шагом 1, равным 1. Если LAST меньше FIRST, он не выдает никаких выходных данных. seq FIRST INCREMENT LAST: когда заданы три аргумента, он генерирует числа от FIRST до LAST на шаге INCREMENT. Если LAST меньше, чем FIRST, он не производит вывод. seq -f «FORMAT» FIRST INCREMENT LAST: эта команда используется для генерации последовательности в форматированном виде. FIRST и INCREMENT являются необязательными. seq -s «STRING» ПЕРВЫЙ ВКЛЮЧЕНО: Эта команда используется для STRING для разделения чисел. По умолчанию это значение равно /n. FIRST и INCREMENT являются необязательными. seq -w FIRST INCREMENT LAST: эта команда используется для выравнивания ширины путем заполнения начальными нулями. FIRST и INCREMENT являются необязательными.

4. Какой результат даст вычисление выражения $\$((10/3))$? Результатом данного выражения

5. Укажите кратко основные отличия командной оболочки zsh от bash. Отличия командной оболочки zsh от bash: В zsh более быстрое автодополнение для cd с помощью Tab В zsh существует калькулятор zcalc, способный выполнять вычисления внутри терминала В zsh поддерживаются числа с плавающей запятой В zsh поддерживаются структуры данных «хэш» В zsh поддерживается раскрытие полного пути на основе неполных данных В zsh поддерживается замена части пути В zsh есть возможность отображать разделенный экран, такой же как разделенный экран vim
6. Проверьте, верен ли синтаксис данной конструкции `for ((a=1; a <= LIMIT; a++))` `for ((a=1; a <= LIMIT; a++))` синтаксис данной конструкции верен, потому что, используя двойные круглые скобки, можно не писать \$ перед переменными ().

7. Сравните язык `bash` с какими-либо языками программирования. Какие преимущества у `bash` по сравнению с ними? Какие недостатки? Преимущества и недостатки скриптового языка `bash`: Один из самых распространенных и ставится по умолчанию в большинстве дистрибутивах Linux, MacOS Удобное перенаправление ввода/вывода Большое количество команд для работы с файловыми системами Linux Можно писать собственные скрипты, упрощающие работу в Linux Недостатки скриптового языка `bash`: Дополнительные библиотеки других языков позволяют выполнить больше действий Bash не является языком общего назначения Утилиты, при выполнении скрипта, запускают свои процессы, которые, в свою очередь, отражаются на скорости выполнения этого скрипта Скрипты, написанные на `bash`, нельзя запустить на других операционных системах без дополнительных действий

Выводы

Мы изучили основы программирования в оболочке ОС UNIX. Научились писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

Список литературы
